

MPI Collective Communications - Algorithms and Communication Times

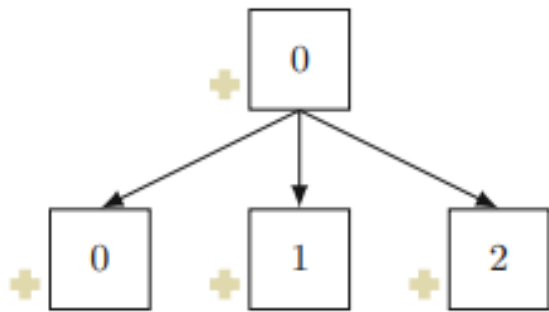
Lecture 15

March 11, 2026

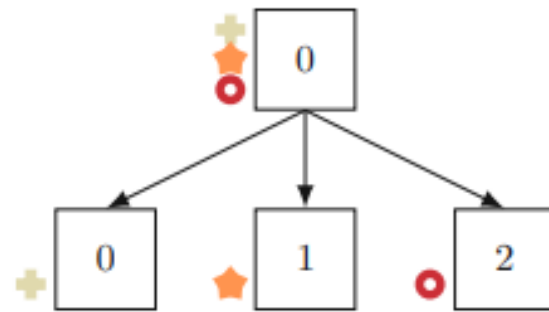
Blocking Collectives

- MPI_Barrier
- MPI_Bcast
- MPI_Gather
- MPI_Scatter
- MPI_Reduce
- MPI_Allgather
- MPI_Allreduce
- MPI_Alltoall

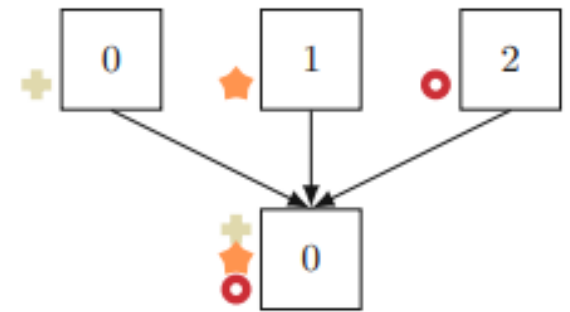
Recap



(a) MPI_Bcast



(b) MPI_Scatter



(c) MPI_Gather

Source: Online Algorithm Selection of MPI Collective Communication Operations, Sebastian et al., 2023

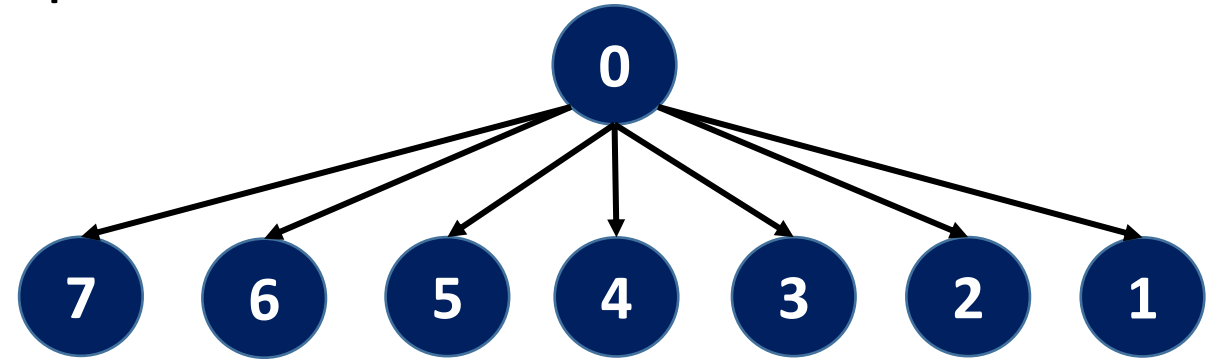
Number of Algorithms

Implementation	Version	Allreduce	Bcast	Alltoall	Allgather
Open MPI	4.1.x	6	9	5	6
Intel MPI	2021.6	12	14	4	5
MPICH	4	8	8	6	5
MVAPICH2	2.3.7	12	14	5	11

Source: Online Algorithm Selection of MPI Collective Communication Operations, Sebastian et al., 2023

Broadcast – Naïve Algorithm

- Root process sends to every other process



- #Steps for p ($=2^d$) processes?
 - $p-1$
- Communication time for n bytes
 - $T(p) = (p - 1) * (L + n/B)$
 - $T(p^2) = (p^2 - 1) * (L + n/B)$

Communication Cost Model

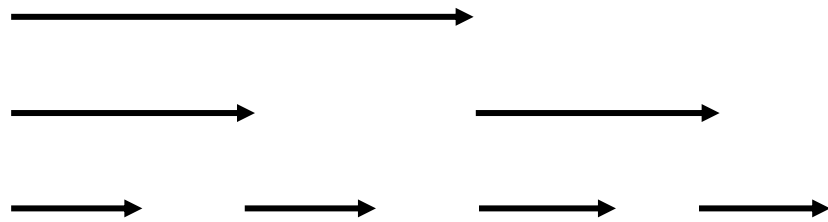
- Communication (or message transfer) time is modeled as $L+n/B$, where L is the latency (or startup time) per message, and $1/B$ is the transfer time per byte, and n is the message size in bytes

Optimization of Collective Communication Operations in MPICH

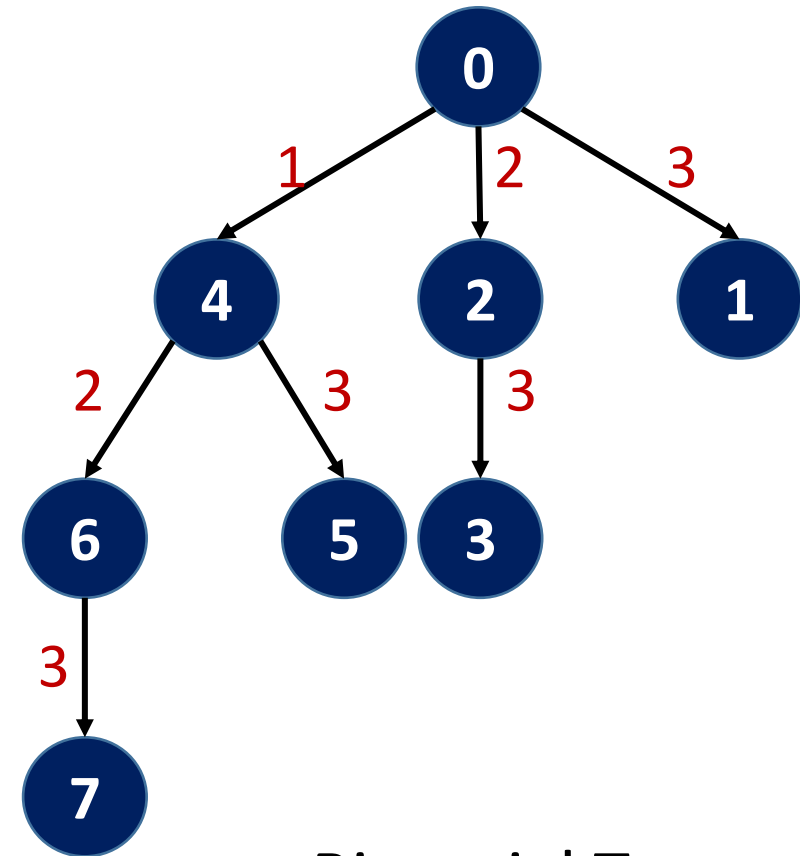
Rajeev Thakur, Rolf Rabenseifner, William Gropp
IJHPCA 2005

Broadcast – Binomial Algorithm

H.W. Write the P2P code



- #Steps for $p (=2^d)$ processes?
 - $\log p$
- Communication time for n bytes
 - $T(p) = \log p * (L + n/B)$
 - $T(p^2) = 2 \log p * (L + n/B)$



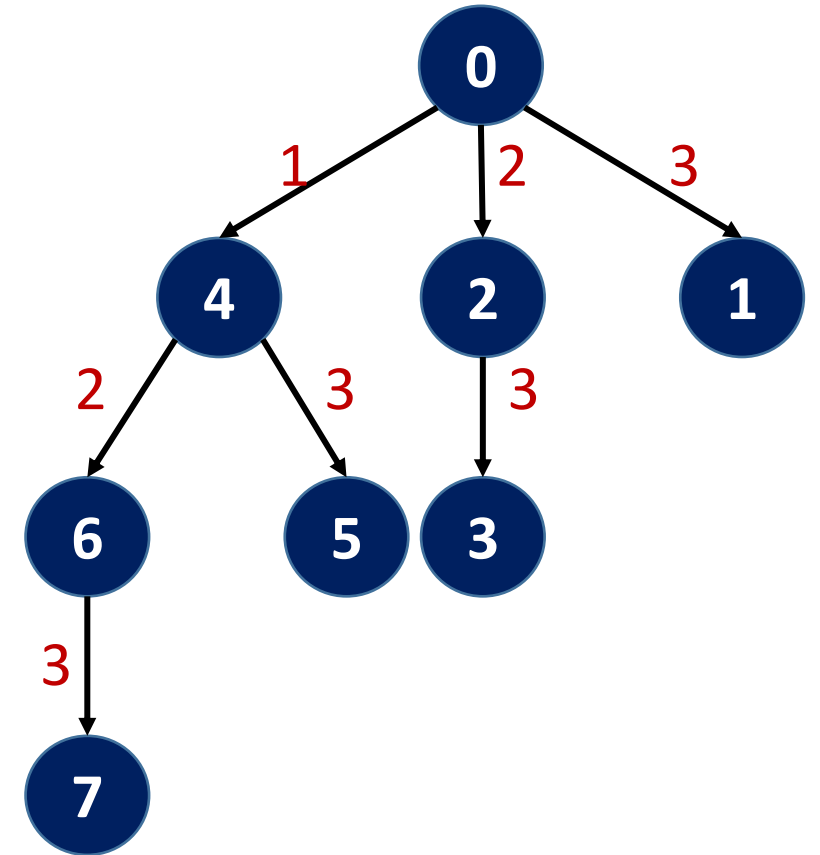
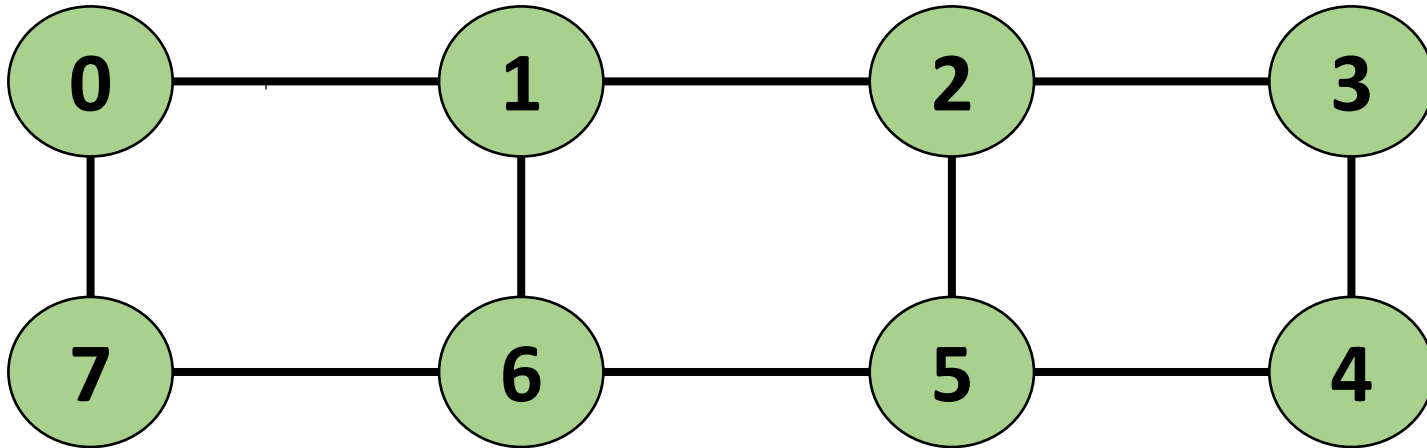
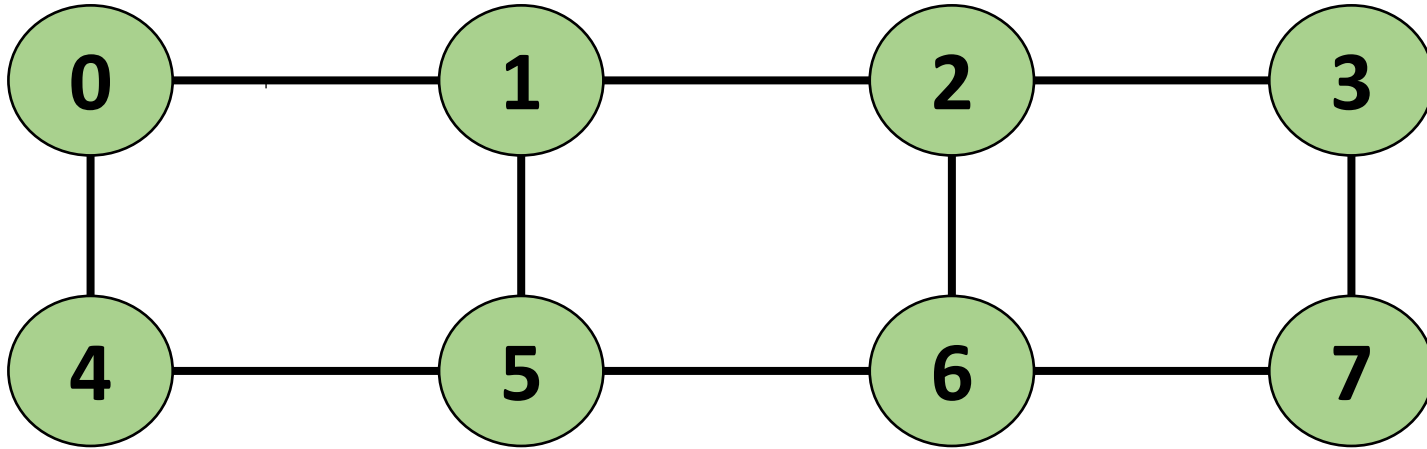
Binomial Tree

MPI_Bcast Timings

```
CS633 $ mpirun -np 12 --hosts csews199:3,csews198:3,csews197:3,csews196:3 ./bcast_scale 100000
0.012307
CS633 $ mpirun -np 12 --hosts csews199,csews198,csews197,csews196 ./bcast_scale 100000
0.015753
CS633 $ mpirun -np 24 --hosts csews199:6,csews198:6,csews197:6,csews196:6 ./bcast_scale 100000
0.011843
CS633 $ mpirun -np 24 --hosts csews199,csews198,csews197,csews196 ./bcast_scale 100000
0.026107
```

Placement of processes and communication pattern together determines the communication performance.

Effect of Process Placement for MPI_Bcast



MPI_Bcast Communications – 16 processes

#1 0 -> 8

#2 0 -> 4 8 -> 12

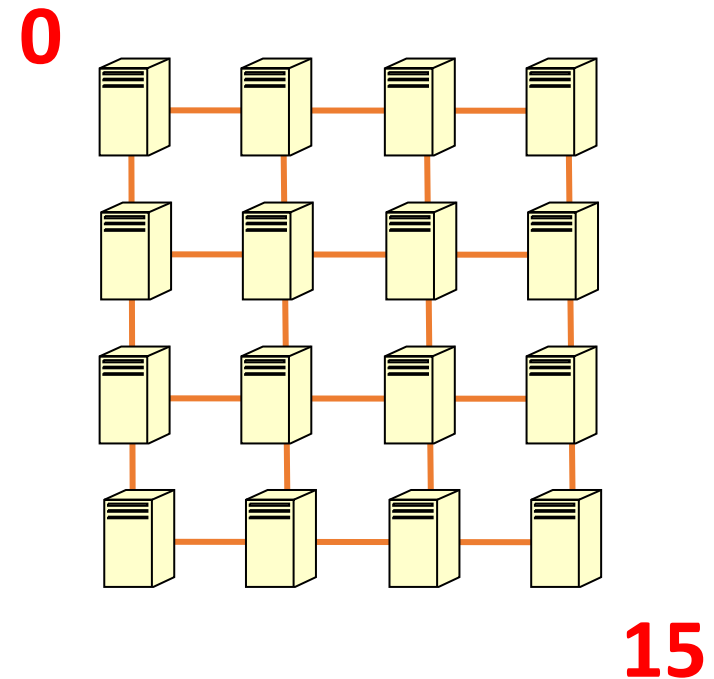
#3 0 -> 2 4 -> 6 8 -> 10 12 -> 14

#4 0 -> 1 8 -> 9
2 -> 3 10 -> 11
4 -> 5 12 -> 13
6 -> 7 14 -> 15

MPI_Bcast Communications on 2D Mesh

#1	0 -> 8 (2)	
#2	0 -> 4 (1)	8 -> 12 (1)
#3	0 -> 2 (2)	4 -> 6 (2)
	8 -> 10 (2)	12 -> 14 (2)
#4	0 -> 1 (1)	8 -> 9 (1)
	2 -> 3 (1)	10 -> 11 (1)
	4 -> 5 (1)	12 -> 13 (1)
	6 -> 7 (1)	14 -> 15 (1)

Total #hops for the above: 2 + 1 + 2 + 1
(considering maximum #hops per round)

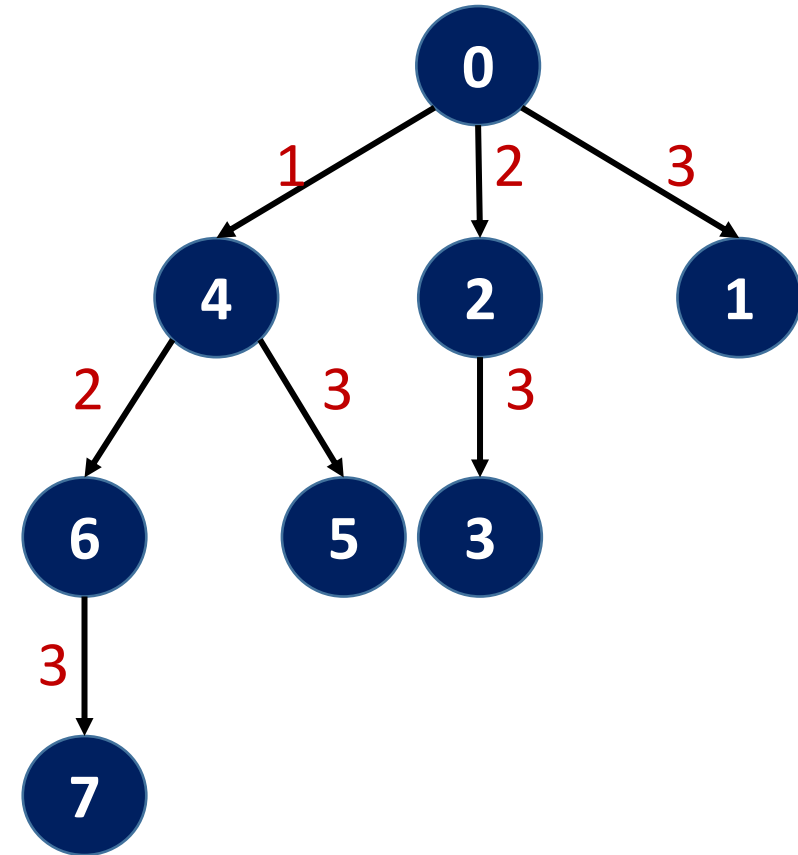


Recap: higher #hops indicate higher communication times

Broadcast

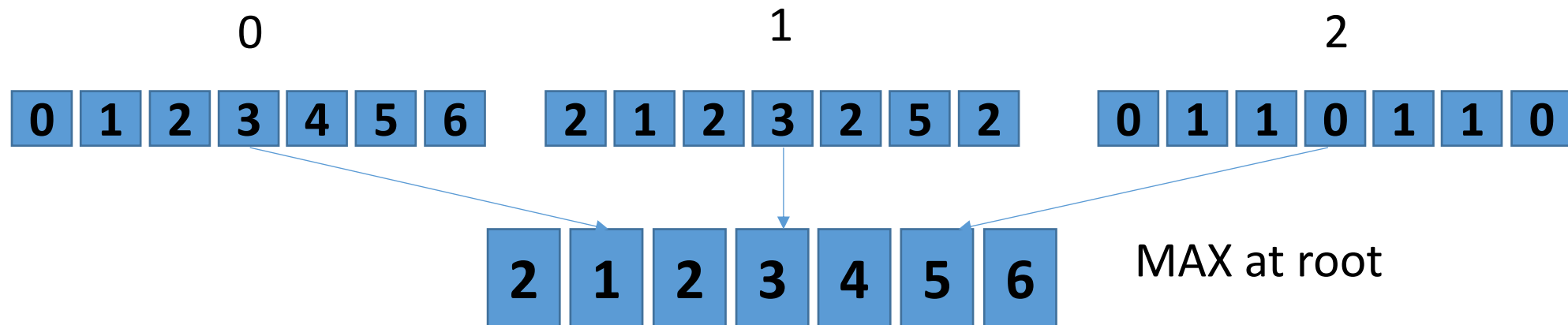
Q: Name one interconnect that may most likely exhibit minimum link contention for binomial tree broadcast algorithm?

Hypercube



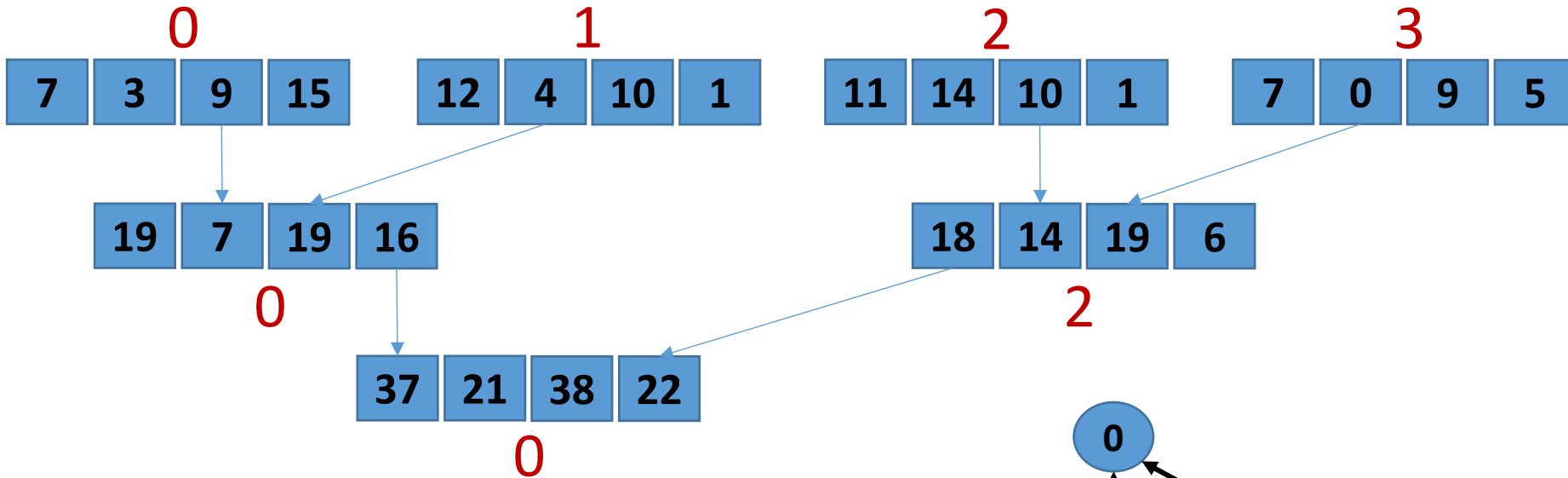
Reduce

- MPI_Reduce (inbuf, outbuf, count, datatype, **op**, root, comm)
- Combines element in inbuf of each process
- Combined value in outbuf of root
- op: MIN, MAX, SUM, PROD, ...
- Note: The operation is done element-wise as shown in the below example



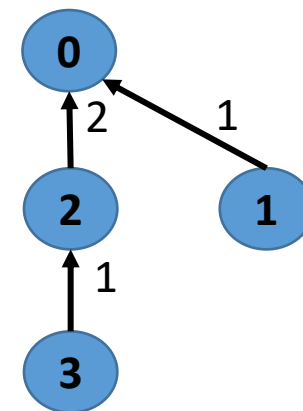
Reduce Algorithm

Recursive (distance) doubling

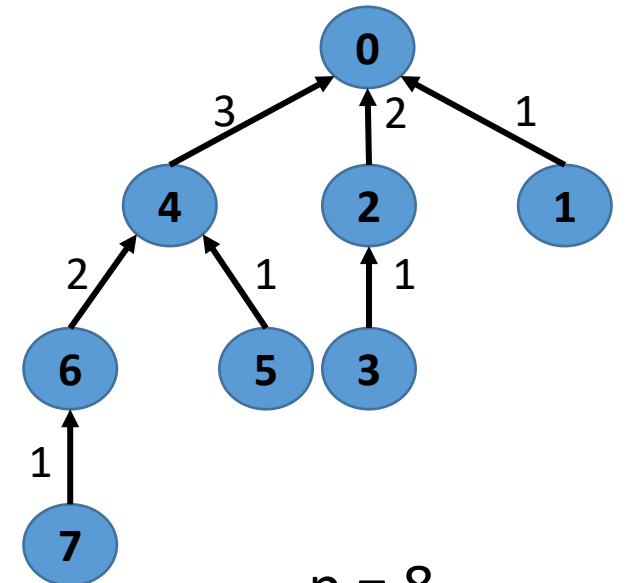


Time: $\log p (L + n*(1/B) + n*c)$
 L = latency, B = bandwidth
 c = compute cost per byte

Used for short messages

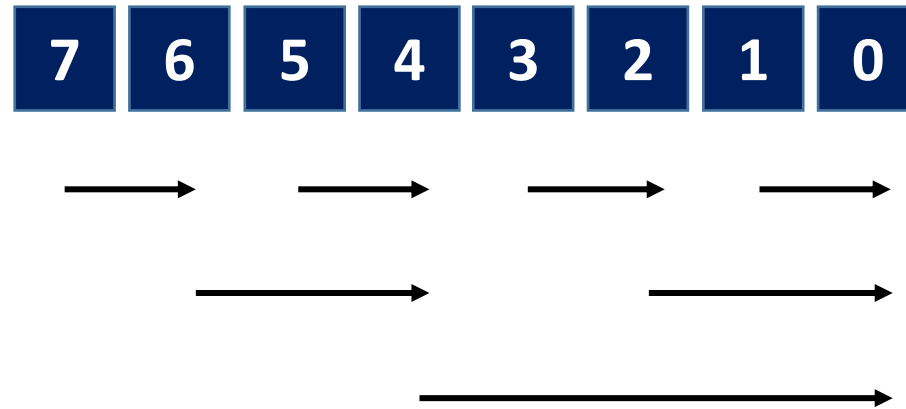


$p = 4$



$p = 8$

MPI_Gather – Recursive Doubling



Vector doubling
Distance doubling

Message size at every process = n

Every step the message size doubles: $n \cdot 2^0, n \cdot 2^1, \dots, n \cdot 2^{((\log p)-1)}$

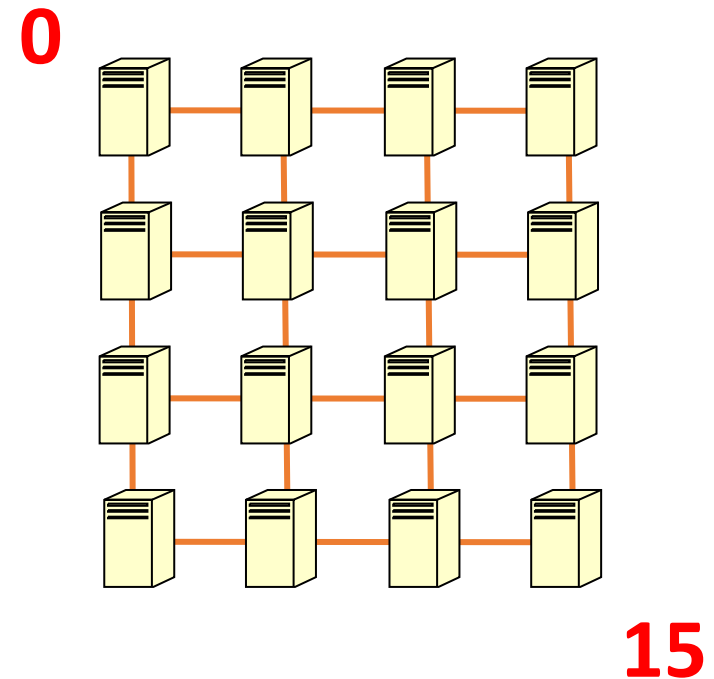
Time for scatter of n bytes from root

$$\log p * L + (p-1)*(n)*(1/B)$$

$$\log p * L + (p-1)*(n/p)*(1/B) \quad [if \text{ Message size} = n/p \text{ at every process}]$$

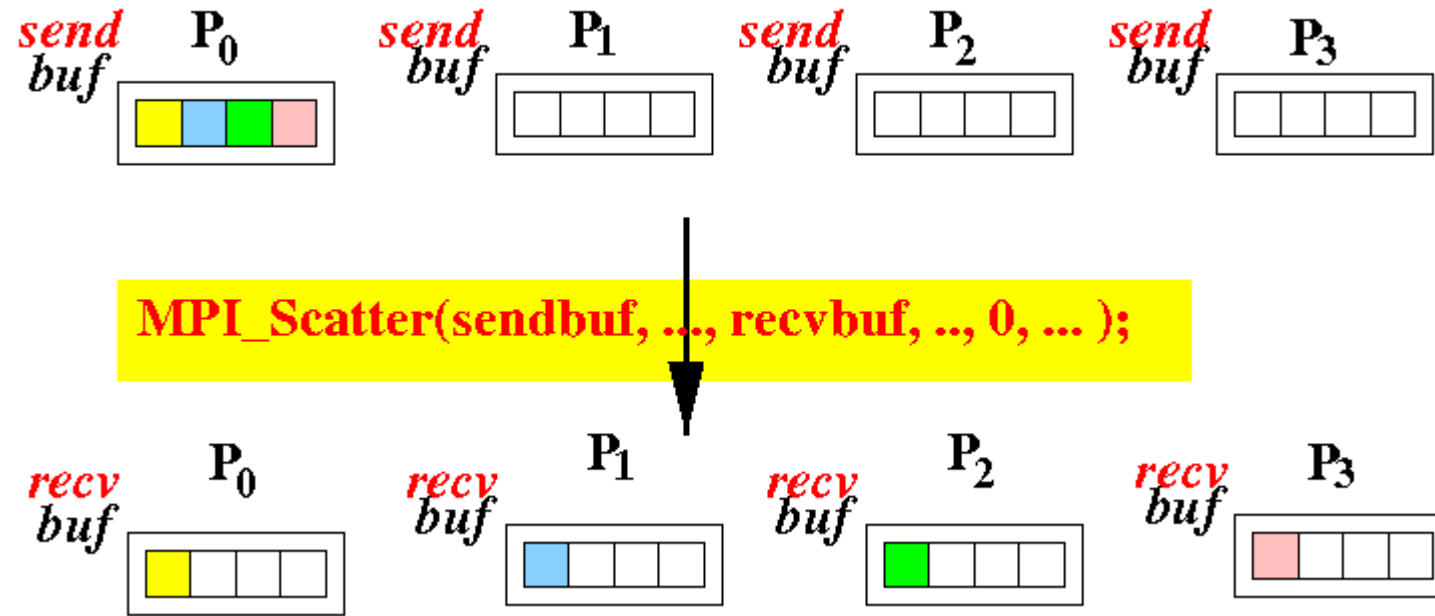
MPI_Gather Communications (Hops, Volume)

#1	$0 \leftarrow 1 (1, n)$	$8 \leftarrow 9 (1, n)$
	$2 \leftarrow 3 (1, n)$	$10 \leftarrow 11 (1, n)$
	$4 \leftarrow 5 (1, n)$	$12 \leftarrow 13 (1, n)$
	$6 \leftarrow 7 (1, n)$	$14 \leftarrow 15 (1, n)$
#2	$0 \leftarrow 2 (2, 2n)$	$4 \leftarrow 6 (2, 2n)$
	$8 \leftarrow 10 (2, 2n)$	$12 \leftarrow 14 (2, 2n)$
#3	$0 \leftarrow 4 (1, 4n)$	$8 \leftarrow 12 (1, 4n)$
#4	$0 \leftarrow 8 (2, 8n)$	

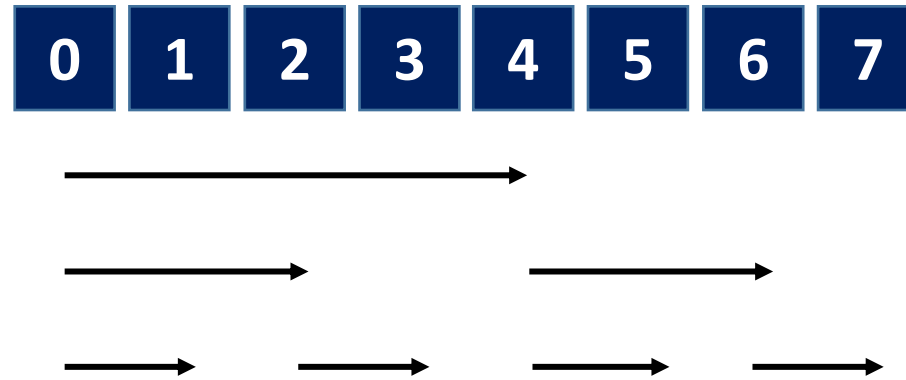


Given: Message size at every process = n ,
default XY process placement

MPI_Scatter Recap



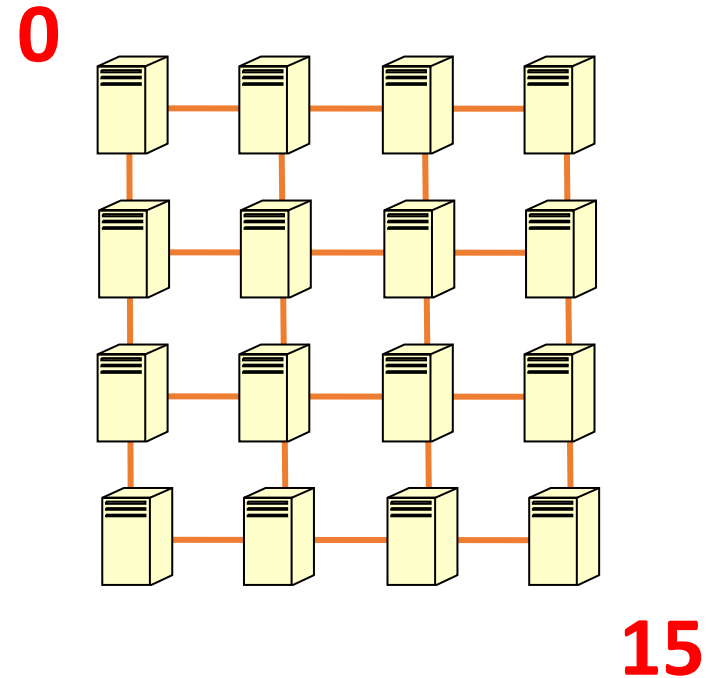
MPI_Scatter



Vector halving
Distance halving

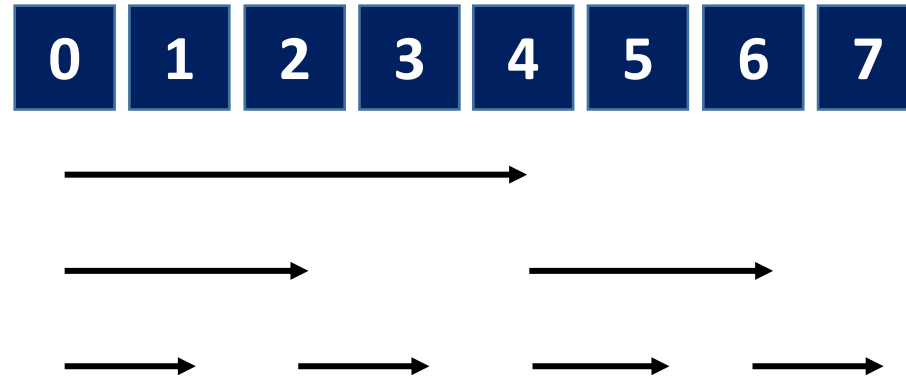
MPI_Scatter Communications

#1 0 -> 8 (2, $n/2$)
#2 0 -> 4 (1, $n/4$) 8 -> 12 (1, $n/4$)
#3 0 -> 2 (2, $n/8$) 4 -> 6 (2, $n/8$)
8 -> 10 (2, $n/8$) 12 -> 14 (2, $n/8$)
#4 0 -> 1 (1, $n/16$) 8 -> 9 (1, $n/16$)
2 -> 3 (1, $n/16$) 10 -> 11 (1, $n/16$)
4 -> 5 (1, $n/16$) 12 -> 13 (1, $n/16$)
6 -> 7 (1, $n/16$) 14 -> 15 (1, $n/16$)



Message size at root = n

MPI_Scatter – Recursive Halving



Vector halving
Distance halving

Message size at root = n

Every step the message size halves: $n/2, n/2^2, \dots, n/2^{(\log p)}$

Time for scatter of n bytes from root
 $\log p * L + (p-1)*(n/p)*(1/B)$

The first term is derived using the number of communication steps and L (latency)
The second term is the summation of all the bandwidth terms ($n/2 * 1/B + n/2^2 * 1/B + \dots + n/2^{(\log p)} * 1/B$)